

**שאלון בחינת גמר**  
**20905 - שפות תכנות**

## משך בחינה: 3 שעות

## בשאלון זה 9 עמודים

## מבנה הבחינה:

בבחינה 3 שאלות.  
משך זמן הבחינה 3 שעות.  
עליכם לענות על כל השאלות.  
משהקל השאלות מפורט בגוף השאלון.

שימו לב:  
את תשובותיכם רשמו אך ורק במחברת הבחינה.  
כתבו בכתב יד ברור ומסודר.

חומר עזר המותר בשימוש:  
ספר הקורס "Essentials of Programming Languages".  
מדריך הלמידה "שפות תכנות".

## חומר עצר:

ספר הקורס: essentials of programming languages  
מדריך הלמידה. מותרות הערות בכתב יד, ע"ג הספרים.  
אסור שימוש בעזרים ובחומרים מקוונים.

## בהצלחה !!!

## אינכם חייבים

## להחזיר את השאלון לאוניברסיטה הפתוחה



## שאלה 1 (35 נקודות)

שאלה זו עוסקת בשדרוג של שפת "וישגור" (שפת PROC) המתוארת בספר הלימוד בפרק 3 בעמודים 74-81.

מעוניינים להרחיב שפה זו בביטוי חדש בשם `forsum-exp` המאפשר לבצע לולאה העוברת על רשימת ביטויים שערכם מספרי, תוך אפשרות לעצירת הלולאה טרם הזמן או דילוג על איברים מהרשימה בתנאים מסויימים (`guards`). הביטוי מחזיר את סכום תוצאות הפעלת גוף הלולאה לכל איבר ברשימה שעליו פעלה הלולאה.

להלן הדקדוק המגדיר את אופן השימוש בביטוי `forsum-exp`:

$Expression ::= \text{for/sum } [ identifier ( \{ Expression \}^{+ (L)} ) ]$

$\{ < guard > \}^*$

$Expression$

`for/end`

|                                               |
|-----------------------------------------------|
| <code>forsum-exp (id exps guards body)</code> |
|-----------------------------------------------|

$guard ::= \text{skip} : Expression$

|                                     |
|-------------------------------------|
| <code>skip-guard ( boolexp )</code> |
|-------------------------------------|

$guard ::= \text{break} : Expression$

|                                      |
|--------------------------------------|
| <code>break-guard ( boolexp )</code> |
|--------------------------------------|

הסבר כללי על אופן פעולת הביטוי החדש :

ביטוי forsum-exp מורכב מהגורמים הבאים :

- משתנה זמני חדש הנתון ע"י id לצורך ביצוע הלולאה.
- מרשימת ביטויים exps אותם משתנה הלולאה אמור לסקור.
- מרשימת guards (יכולים להיות מ-2 סוגים אפשריים : skip או break ויכולים לחזור על עצמם) כל סוג guard מורכב מביטוי שערכו בוליאני.
- מביטוי body שהוא גוף הלולאה.

הלולאה אמורה לעבור באמצעות משתנה הלולאה id על איברי הרשימה exps, החל מהאיבר הראשון, ולכל איבר לבדוק תחילה אם מתקיים עבורו אחד מה-guards (יבדקו לפי סדר הגדרתם), במידה ואין guards או שאף אחד מהם אינו מתקיים, יבוצע גוף הלולאה עבור איבר זה, ותוצאת גוף הלולאה תתווסף לסכום הסופי שיוחזר. במידה ואחד ה-guards התקיים עבור אותו איבר, אזי גוף הלולאה לא יבוצע עבור אותו איבר, אם ה-guard שהתקיים הוא מסוג skip יש להמשיך את הלולאה עם יתר האיברים, ואם ה-guard שהתקיים הוא מסוג break, יש להפסיק את הלולאה ולהחזיר את הסכום שהצטבר עד לשלב זה.

להלן מספר דוגמאות להמחשת השימוש בביטוי החדש :

#### דוגמה 1:

```
> (run "for/sum [i (1,2,3,20,5,90)]
    <skip: zero? (- (i,1))>
    <break: zero? (- (i,5))>
    <skip: zero? (- (i,3))>
    - (i,-5)
    for/end")
(num-val 32)
>
```

#### הסבר דוגמה 1:

בדוגמה 1, מתוארת לולאה עם משתנה בשם i ורשימת ביטויים שערכם מספרי. ללולאה 3 guards, הבודקים אם איבר הוא 1 או 3 יש לדלג עליו, ואם איבר הוא 5 יש להפסיק את הלולאה. לפיכך הלולאה תפעל רק על האיברים 2 ו-20 (ב-5 היא כבר תעצור) לכל אחד מהאיברים עליהם פועלת הלולאה, מבוצע גוף הלולאה שמחזיר את ערכו של משתנה הלולאה באותו רגע בתוספת 5. לכן הסכום הכולל שיצטבר יהיה 32 (  $20+5 + 2+5$  ) ולכן ערכה של התוכנית הוא 32.

## דוגמה 2:

```
> (run "for/sum [i ()]
      <skip: zero? (- (i,1))>
      <break: zero? (- (i,5))>
      <skip: zero? (- (i,3))>
      - (i,-5)
    for/end")
⊗ ⊗ interp.scm:79:16: forsum-exp: The list for i is empty
> |
```

## הסבר דוגמה 2:

בדוגמה 2, מודגמת לולאה עם רשימת ערכים ריקה, זה אינו תקין תחבירית (על פי הדקדוק הנתון) ולכן הודפסה הודעת שגיאה מתאימה

## דוגמה 3:

```
> (run "for/sum [i (1,2,3,20,5,90)]
      - (i,-5)
    for/end")
(num-val 151)
>
```

## הסבר דוגמה 3:

בדוגמה 3, מודגמת לולאה שאינה כוללת guards (זה תקין תחבירית) ולכן הלולאה פועלת על כל האיברים ומוחזר הסכום המצטבר של פעולת גוף הלולאה על איברי הרשימה.

**ממשו והטמיעו** את השינויים הדרושים כדי שהוגדרו והודגמו בתוך שפת **"וישגור" (שפת PROC)**. בתשובתכם, הקפידו להסביר **היכן בדיוק** נדרשים שינויים ותוספות בקבצי המפרש, **ומהם** השינויים והתוספות.

## הנחיות כלליות לפתרון:

- 1) הקפידו על כתב ברור, ועל קוד מבני ומסודר.
- 2) הקפידו על פתיחת וסגירת סוגריים במקומות הנכונים
- 3) הפתרון אמור לשמור על הגדרות השפה המקוריות (פרט למקרים בהם נדרש שינוי כזה במפורש בשאלה)
- 4) על מנת לפשט קוד ארוך, כדאי ורצוי להיעזר בכתיבת פרוצדורות עזר (מחוץ ל-value-of או כאלה המוטמעות בתוכו)
- 5) בפתרונכם, הקפידו על אבחנה ושימוש נכון בין ביטוי (expression) לבין תוצאת חישוב ביטוי (expval) לבין identifier
- 6) ניקוד יורד על אי ניתוח ומימוש נכון של הדקדוק הנתון בשאלה, כלומר יש להפקיד על זיהוי וטיפול נכון ב- terminals, non-terminals ופעולות של סגור ( { } \* ) או + { }

## שאלה 2 (35 נקודות)

בספר הלימוד בעמודים 113-120 מתוארת שפת "וירמוז" (IMPLICIT-REFS).

ברצוננו להרחיב שפה זו עם יכולת של הגדרת מצביע רב מימדי, וגישה באמצעותו לנתונים.

להלן הדקדוק המגדיר את הביטויים החדשים שיש להטמיע בשפה:

$Expression ::= \{ \& \}^+ (Expression)$

**multiptr-exp (ptrs data)**

$Expression ::= \{ * \}^+ (Expression)$

**multistar-exp (stars v)**

כמו כן, יבוצע עדכון של ביטוי assign-exp (פעולת set) הקיים בשפת "וירמוז" כך שימשיך לפעול כרגיל ובנוסף יהיה אפשר לעדכן דרכו גם את ערכו של נתון המוצבע ע"י מצביע רב מימדי. לכן נעדכן את התחביר (וגם ההתנהגות) של ביטוי assign-exp על פי הדקדוק הבא:

$Expression ::= option Identifier = Expression$

**assign-exp (opt id newval)**

$Option ::= \epsilon \mid @$

### הסבר על מרכיבי הביטויים ואופן פעולתם:

לביטוי assign-exp המעודכן התווסף מרכיב של **option**. מרכיב זה או שהוא הסימן @ או שאינו מופיע כלל (מחרוזת ריקה זה מה שמציין  $\epsilon$ ). כאשר option זו מחרוזת ריקה אזי הביטוי assign-exp כתוב ומתפקד כמו בשפת וירמוז המקורית. ובמקרה ו-option הוא התו @, רכיב ה-identifier חייב להיות שם של משתנה שערכו הוא מסוג מצביע רב מימדי לעבר נתון, ובמקרה זה ביטוי assign-exp יעדכן את ערכו של הנתון להיות לפי ערכו של הביטוי המיוצג ע"י newval.

ביטוי **multiptr-exp** מאפשר הגדרת מצביע רב מימדי (מצביע למצביע) על נתון מסוים. הביטוי ptrs מייצג רשימה של איברים שהם התו &, אורכה של הרשימה מציין את המימדיות של המצביע.

מרכיב data הוא ביטוי שעל ערכו מצביעים בצורה רב מימדית. ביטוי **multiptr-exp** אמור להחזיר expval מסוג חדש המייצג ישות של ערך המוצבע בצורה רב מימדית.

**ביטוי star-exp** מאפשר פניה אל הנתון המוצבע ע"י המצביע הרב מימדי המיוצג ע"י הביטוי  $v$ , או אל אחד המצביעים בדרך אל הנתון (שגם הוא ישות מסוג מצביע רב מימדי לנתון). הביטוי מורכב מרשימה של  $*$  (כוכביות). אורכה של הרשימה מציין את כמות המצביעים שנדרש בעזרתם להתקדם לעבר הנתון עצמו, הערך המוחזר הוא : הנתון עצמו במקרה והגענו עד אליו, או מצביע רב מימדי לנתון אליו התקרבו אך עדיין מצביעים עליו, או שגיאה במקרה והתקדמנו יותר מדי (מעבר למימד של המצביע הרב מימדי)

הבהרה : את ביטוי star-exp אפשר ליישם רק על ביטוי מסוג **multiplr-exp**, כל ניסיון ליישמו על ביטויים אחרים בשפה אמור לייצר שגיאה.

להלן מספר דוגמאות שימוש.

### דוגמה 1:

```
let p = &&&(6)
in
  let q = **(p)
  in
    let num = *(q)
    in
      -(num, 2)
```

בדוגמה זו מוגדר מצביע תלת מימדי  $p$  ( כלומר, מצביע למצביע למצביע ל- 6).

המצביע  $q$  הוא מצביע חד מימדי המצביע לעבר 6. ו- $num$  הוא 6

לכן ערכו של הביטוי הוא  $(num-val\ 4)$ .

### דוגמה 2:

```
let p = &&&(6)
in
  let q = **(p)
  in
    let num = ****(q)
    in
      -(num, 2)
```

בדוגמה זו מוגדר מצביע תלת מימדי  $p$  ( כלומר, מצביע למצביע למצביע ל- 6).

המצביע  $q$  הוא מצביע חד מימדי המצביע לעבר 6. ב-  $let$  השלישי יש ניסיון לעבוד עם  $q$  שהוא מצביע חד מימדי, ולהתקדם בעזרתו 4 מצביעים (שלא קיימים) לעבר נתון ולכן אמורה להיות מודפסת הודעת שגיאה על שימוש שגוי במצביע.

### דוגמה 3:

```
let p = **(&&&(6))
in
  -( *(p), 5)
```

בדוגמה זו  $p$  הוא מצביע חד מימדי על הנתון 6.

בגוף ה- $let$  מתבצעת פניה אל 6 באמצעות המצביע החד מימדי (עם כוכבית אחת עליו) מה שמחזיר 6, ולכן ערכו של כל הביטוי הוא  $(num-val\ 1)$

#### דוגמה 4:

```
let p = *( zero?( 6))
in
  9
```

בדוגמה זו יש ניסיון להפעיל ביטוי star-exp על ביטוי מסוג zero?-exp שאינו מייצג מצביע רב מימדי ולכן אמורה להתקבל שגיאה מתאימה.

#### דוגמה 5:

```
let p = &&&(proc (x) -(x,7))
in
  (**(p) 9)
```

בדוגמה זו p הוא מצביע תלת-מימדי לפרוצדורה, בגוף ה-let יש פניה אל הפרוצדורה והפעלתה על 9 ולכן תוצאת התוכנית היא (num-val 2)

#### דוגמה 6:

```
let p = &&&(proc (x) -(x,7))
in
  *(p)
```

בדוגמה זו p הוא מצביע תלת-מימדי לפרוצדורה, מגוף ה-let מוחזרת תוצאה המייצגת מצביע דו-מימדי לפרוצדורה. ולכן תוצאת התוכנית אמורה להיות expval המייצג מצביע רב מימדי

#### דוגמה 7:

```
let p = &&&(proc (x) -(x,7))
in
  begin
    set @p = 9;
    -(**(p) , 2)
  end
```

בדוגמה זו p הוא מצביע תלת-מימדי לפרוצדורה, בגוף ה-let מעודכן המצביע התלת מימדי להיות מצביע על 9 במקום מצביע על פרוצדורה, ולבסוף מוחזר 9-2 שהוא (num-val 7)

### דוגמה 8:

```

let p = &&&&(8)
  in
  let q = **(p)
  in
  begin
    set @q = 20;
    -(****(p), 3)
  end

```

בדוגמה זו, p הוא מצביע 4-מימדי על 8, q הוא מצביע דו-מימדי לאותו 8, בגוף בלוק ה- begin משנים את ערכו של המצביע הדו מימדי (8) ל-20, ואז נעשית פנייה לערכו של המצביע ה-4 מימדי p ומתקבל 20. ולכן התוכנית מחזירה (num-val 17)

### דוגמה 9:

```

let p = &&&&(8)
  in
  let q = **(p)
  in
  begin
    set @p = 20;
    set p = 5;
    -(** (q), p)
  end

```

בדוגמה זו, p הוא מצביע 4-מימדי על 8, q מצביע דו מימדי לאותו 8. ערכו של הנתון המוצבע ע"י המצביע p משתנה מ-8 ל-20 (מה שמשפיע גם על q), לאחר מכן, ערכו של p עצמו משתנה להיות 5 (הוא כבר לא ישות של מבציע רב מימדי, אך q כן ממשיך להיות מצביע דו-מימדי), לבסוף מוחזר ההפרש בין ערכו של הנתון המוצבע ע"י q (שהוא 20) לבין ערכו של p (שכעת הוא 5), ולכן ערכה של כל התוכנית הוא (num-val 15)

**ממשו והטמיעו** את השינויים הדרושים כדי שהוגדרו והודגמו בתוך שפת **"וירמוז"** (שפת **Implicit- (Refs)**).

בתשובתכם, הקפידו להסביר **היכן בדיוק** נדרשים שינויים ותוספות בקבצי המפרש, **ומהם** השינויים והתוספות.

### הנחיות כלליות לפתרון:

- 1) הקפידו על כתב ברור, ועל קוד מבני ומסודר.
- 2) הקפידו על פתיחת וסגירת סוגריים במקומות הנכונים
- 3) הפתרון אמור לשמור על הגדרות השפה המקוריות
- 4) על מנת לפשט קוד ארוך, כדאי ורצוי להיעזר בכתיבת פרוצדורות עזר (מחוץ ל-value-of או כאלה המוטמעות בתוכו)
- 5) בפתרונכם, הקפידו על אבחנה ושימוש נכון בין ביטוי (expression) לבין תוצאת חישוב ביטוי (expval) לבין identifier
- 6) ניקוד יורד על אי ניתוח ומימוש נכון של הדקדוק הנתון בשאלה, כלומר יש להפקיד על זיהוי וטיפול נכון ב- terminals, non-terminals ופעולות של סגור ( { } \* ) או + ( { }



### שאלה 3 (30 נקודות)

להלן תוכנית בשפת Implicit-Refs (שפת "וירמוז")

```
> (run
  "let y=12
    in
      let x= proc(x) -(x,1)
      in
        begin
          set y = (proc (x) (x 3) x);
          y
        end")
```

תוצאת הריצה של תוכנית זו היא :

(num-val 12) א)

(num-val 9) ב)

התוכנית מחזירה שגיאה בזמן הריצה. ג)

(num-val 11) ד)

(num-val 2) ה)

(num-val 4) ו)

(proc-val ...) ז)

נמקו והסבירו את תשובתכם ע"י הסבר של דרך חישוב תוצאת התוכנית.

את תשובתכם רשמו במחברת הבחינה בלבד.

**בהצלחה !**